Reflective Hybrid RAG for Recommendation

Sherine Chally

Abstract

Modern Large Language Model (LLM)-based recommendation systems typically suffer from **Context Fragmentation**. When relying on the naïve retrieval of raw user history, the model is forced to reconstruct a user's preference profile from scratch during every inference cycle using noisy, low-level logs. This limitation restricts personalization depth and increases computational overhead. We introduce a **Reflective Hybrid RAG** architecture that separates historical "facts" from synthesized "wisdom." By combining **Hybrid Search** (Dense + Sparse SPLADE) with a novel **"Sidecar" Reflection Layer**, we enable the agent to infer high-level user intent. Experimental results on the Agent Society Challenge dataset show that our Reflective agent achieves a **3x improvement in Hit Rate @ 1 (37.50%)** compared to a baseline LLM recommender (12.50%), validating that metacognitive memory is a critical component for next-generation recommendation agents.

1. Introduction

1.1. The Context

The integration of Large Language Models (LLMs) into recommendation systems has revolutionized how agents process textual data. Traditional collaborative filtering relies on implicit signals (clicks/ratings), but LLMs can interpret explicit feedback (reviews) to understand *why* a user liked an item. However, most existing approaches rely on simple "prompting" or basic RAG pipelines that retrieve raw interaction logs without synthesis.

1.2. The Problem

Raw history is noisy and fragmented. When an agent is fed a list of isolated events—e.g., *"User rated Book X 5 stars," "User rated Book Y 2 stars"*—it forces the model to re-infer the user's taste from scratch for every single prediction. This process is computationally inefficient and prone to **reasoning drift**, where the model fixates on recent, potentially anomalous behavior rather than the user's long-term identity

1.3. The Solution

We propose a **Reflective Hybrid Agent** that utilizes a dual-memory architecture:

1. **Fact Memory:** Hybrid retrieval of specific historical interactions.

2. **Wisdom Memory (Sidecar):** A synthesized "User Profile" generated offline that captures high-level traits (e.g., *"User prefers slow-burn sci-fi over action-heavy fantasy"*).

2. Methodology: The Reflective Hybrid Architecture

Our system follows a **Retrieve-Reflect-Reason** workflow designed to minimize hallucination and maximize personalization.The system is structured around three components: Memory, Reflection, and Reasoning.

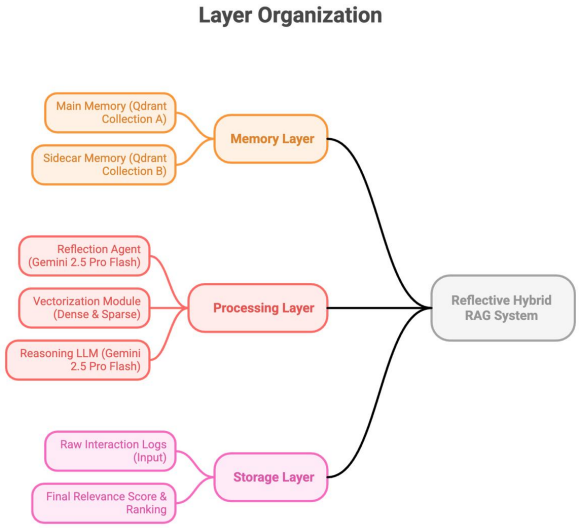

Figure 2. Layer Organization of the Reflective Hybrid RAG System

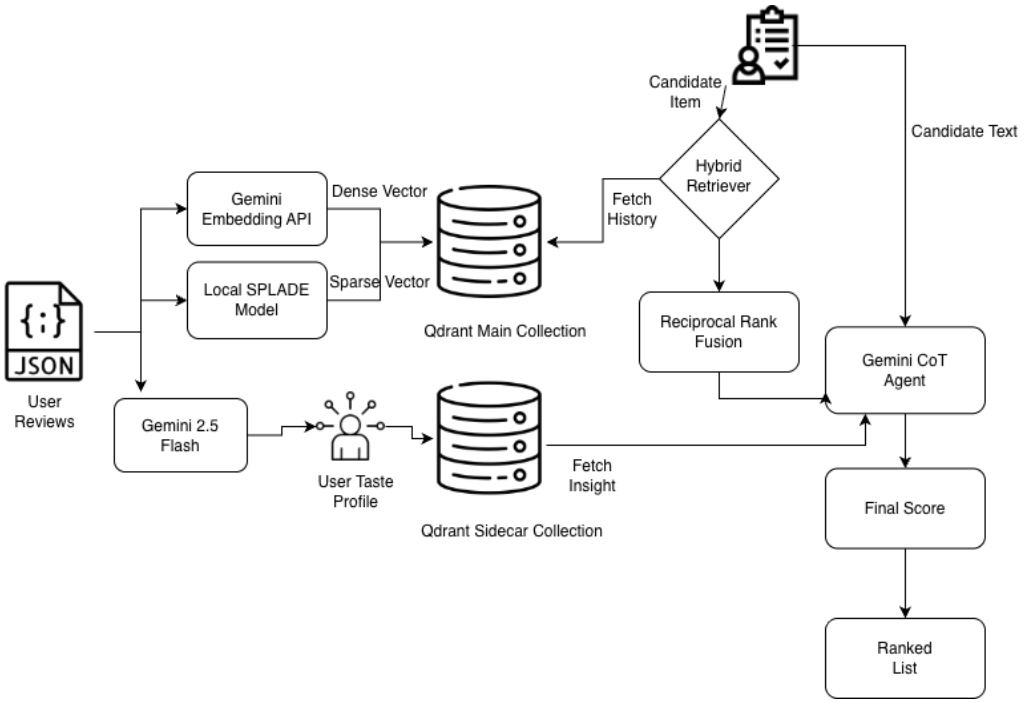

Figure 1. Workflow of Reflective Hybrid RAG

2.1. The Memory Layer (Qdrant)

We use the popular open source vector database Qdrant to store our collections, the primary being, `goodreads_youngadult_reviews_gemini`, which stores user reviews and item metadata post EDA and data preparation.

Hybrid Search Strategy: We utilize **Hybrid Search** to overcome the limitations of single-vector retrieval:

- **Dense Vectors (Semantic):** We embed reviews using Gemini's `text-embedding-004` (768 dimensions) to capture conceptual similarity (e.g., matching "space opera" with "galactic war").
- **Sparse Vectors (Keyword):** We generate sparse vectors using the **SPLADE** model to ensure exact keyword matching (e.g., specific author names or sub-genre terms like "Vampire").
- **Fusion:** These signals are combined using **Reciprocal Rank Fusion (RRF)**, ensuring that retrieved context is both conceptually relevant and lexically precise.

2.2. The Reflection Layer (The Sidecar)

Apart from the hybrid vector search , this is what adds novelty to our architecture. The "Sidecar" is a secondary memory collection containing synthesized insights. Instead of relying solely on raw logs, we implement an offline "Reflection Loop":

- Input: Raw Review History (e.g., "User liked Hunger Games, User liked Divergent").
- Synthesis Process: A high-intelligence LLM (Gemini 2.5 Pro Flash) analyzes the history to extract distinct traits, deal-breakers, and aesthetic preferences of the user.
- Output: A concise **"Taste Profile"** in natural language (e.g., "User prefers dystopian novels with strong female protagonists and avoids romance-heavy plots").
- Storage: This insight is embedded and stored in a "Sidecar" collection `user_insights_sidecar`. This profile acts as a "Cheat Sheet" for the inference agent.

2.3. The Reasoning Layer (Chain-of-Thought using Gemini)

The final decision is made by a **Chain-of-Thought (CoT)** agent. The prompt is structured to force a multi-step reasoning process:

- **Analyze Profile:** Read the Sidecar Insight to understand the user's identity.

Figure 3. Workflow of the Reflection and Reasoning layer

- **Verify Evidence:** Check the Raw History to confirm if the specific candidate item fits this identity.
- **Score:** Assign a likelihood score (0.0 - 1.0).

The agent is prompted to "think step-by-step" about the alignment between the user's known preferences and the item's attributes before assigning a ranking score.

3. Experiment Setup

We evaluate the system on the Goodreads Young Adult subset.

3.1. Dataset Acquisition and Preprocessing

The Goodreads Young Adult dataset was used for this project, given its rich textual reviews and specific sub-genre nuances (e.g., Dystopian vs. Fantasy). The raw data, consisting of compressed JSON logs (`goodreads_books_young_adult.json.gz` and `goodreads_reviews_young_adult.json.gz`) was studied, cleansed and prepared for optimal usage.

Preprocessing Pipeline: The preprocessing script (`process_my_data.py`) parses the raw GZIP files and restructure them into the standard Entity-Relation format required by the Agent Society Simulator:

1. Item Extraction: Parsing book metadata to generate item.json.
2. User Extraction: Anonymizing and aggregating user profiles into user.json.
3. Review Extraction: Formatting interaction logs into review.json.
4. Subset Creation: The full dataset contains over 2.3 million reviews, which presents significant challenges regarding API rate limits and embedding costs. Hence we implemented a subsetting utility (`create_subset.py`). This module extracts the first 50,000 reviews chronologically. This subset size was selected to balance statistical significance with computational feasibility.

3.2. Hybrid Index Construction (Fact Layer)

The core retrieval system relies on a Qdrant vector database populated via a high-throughput ingestion script `ingest_data_optimized.py`. This stage transforms the raw textual data into a dual-vector format to leverage Hybrid Retrieval.

- **Dense Embeddings**: For semantic retrieval, we employed Google's text-embedding-004 model. We utilized a batch processing strategy (Batch Size = 50) to optimize throughput. Each review text was embedded into a 768-dimensional vector designed to capture conceptual similarity (e.g., associating "wizards" with "fantasy").
- **Sparse Embeddings**: To address the "lexical gap" inherent in dense retrievers, we locally implemented the SPLADE (Sparse Lexical and Expansion) model. Leveraging the Hugging Face Transformers library and PyTorch, we computed sparse vectors on the CPU to extract precise keyword weights.
- **Storage**: We encapsulated both dense and sparse vectors within a single Qdrant `PointStruct` using named vectors, a configuration that facilitates simultaneous hybrid querying. Given the computational cost of generating dual embeddings for the 50,000-document corpus, this ingestion step was executed as a one-time offline process.

3.3. Generative Reflection Pipeline (The Sidecar Layer)

A defining characteristic of our architecture is the "Sidecar" memory, which necessitates the pre-calculation of user insights prior to inference. This process is orchestrated by the `generate_reflections.py` module and consists of four stages:

The Reflection Loop:

- History Aggregation: We query the processed dataset to aggregate the complete interaction history for every user in the test set, filtering for sufficient activity ($N \geq 2$).

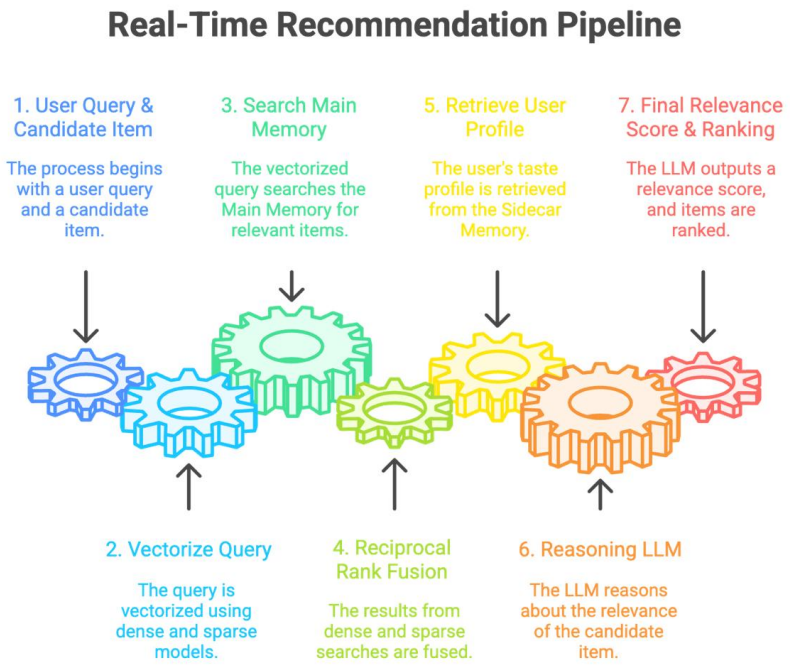

Figure 4. Real-Time Recommendation Pipeline

- Metacognitive Synthesis: The system prompts `Gemini 2.5 Pro Flash` to analyze the aggregated reviews. The prompt is engineered to abstract away surface-level details and focus on metacognitive traits, explicitly asking: *"What are the underlying themes? What is the user's preferred writing style?"*
- Vectorization: The resulting natural language insights (e.g., *"User prefers slow-burn romance over action-heavy plots"*) are encoded into dense vectors using the `text-embedding-004` model.
- Segregated Indexing: These vectors are upserted into the dedicated Qdrant collection (`user_insights_sidecar`). This creates a secondary semantic index that is queryable in parallel with, but architecturally distinct from, the raw review index.

3.4. Evaluation Protocol and Task Generation

To benchmark the agent's performance, we implemented a **"Leave-One-Out"** evaluation strategy, codified in the `generate_tasks.py` module. The task generation protocol proceeds as follows:

- Ground Truth Selection: For every user in the evaluation pool, we hold out the most recent interaction (the last book reviewed) to serve as the positive Ground Truth target.
- Negative Sampling: To simulate a realistic ranking scenario, we randomly sample 19 negative items, books with no recorded interaction in the user's history from the global item corpus.
- Candidate Set Construction: The positive target is merged with the 19 negatives to create a fixed candidate list of 20 items. This list is randomly shuffled to eliminate positional bias during LLM inference.
- Data Filtering: We enforce a strict quality filter, excluding users with sparse history ($N \geq 2$) or insufficient negative samples, ensuring valid historical context is available for the reasoning agent.

This process yields a standardized evaluation dataset (`test.json`), providing a consistent baseline for measuring Hit Rate across different agent architectures.

3.5. Evaluation of Agents

The final evaluation was conducted using the `run_comparison.py` module, which orchestrates the simultaneous execution of three distinct agent configurations on the generated test suite.

System Configuration:

- Orchestration: Python 3.10 environment utilizing

`concurrent.futures` for parallelized agent execution (up to 8 workers).

- Vector Database: Qdrant Cloud (Standard Tier) used for both the Main (Raw) and Sidecar (Reflection) collections.
- Inference Engine: Gemini 2.5 Pro Flash (via Google GenAI SDK) served as the core reasoning engine and dense embedding generator `text-embedding-004`.
- Sparse Embedding: Local CPU-based inference using the `transformers` library and PyTorch to generate SPLADE sparse vectors (`efficient-splade-VI-BT-large-query`).

Agent Benchmarks: We compared the following architectures to isolate the impact of each component:

- **Baseline Agent (Zero-Shot)**: A standard LLM recommender that relies solely on the provided candidate metadata and its internal parametric knowledge, operating with no external retrieval context.
- **Hybrid V1 (Raw RAG)**: Utilizing the Hybrid Search index (Gemini Dense + SPLADE Sparse), this agent retrieves specific historical user reviews but lacks the synthesized "Sidecar" memory layer.
- **Hybrid V2 (Reflective RAG)**: The complete architecture, augmenting the Hybrid Search facts with the Sidecar index to retrieve pre-computed "Taste Profiles" for metacognitive reasoning.

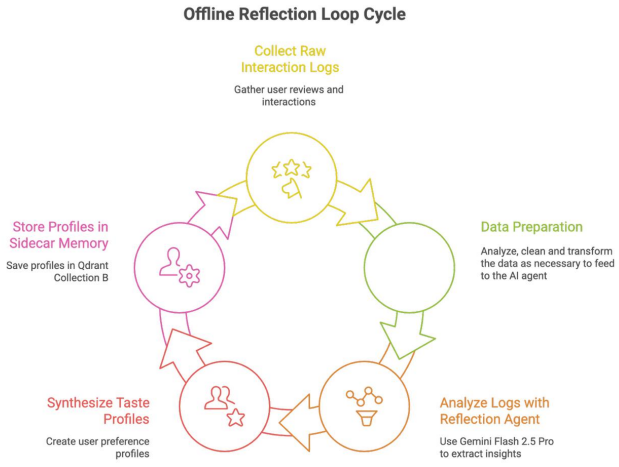

Figure 5. Offline Reflection Loop Cycle

4. Experiment Analysis

4.1. Quantitative Performance

The Reflective Hybrid Agent (V2) demonstrated superior performance across all ranking metrics, validating the efficacy of our proposed architecture.

We evaluated three agent configurations: a **Baseline** (Zero-Shot LLM), **Hybrid V1** (Raw RAG), and **Hybrid V2** (Reflective RAG). To provide a comprehensive assessment of ranking quality, we measured performance using **Hit Rate (HR@K)**, **Normalized Discounted Cumulative Gain (NDCG@10)**, and **Mean Reciprocal Rank (MRR)**.

Key Findings:

- The Value of Retrieval (V1 vs. Baseline): Adding Hybrid Search doubled the HR@1. The baseline often hallucinated user preferences or defaulted to generic popularity (e.g., always recommending "The Hunger Games"). Providing historical context grounded the model.
- The Value of Reflection (V2 vs. V1): The addition of the Reflection Layer yielded a further 50% improvement over V1. This confirms our hypothesis that raw history is insufficient; the synthesis of that history into a coherent persona is required for high-precision ranking.

4.2. Ablation Study

To isolate the contribution of specific architectural components, we analyzed the marginal performance gains of adding Hybrid Retrieval and the Sidecar Reflection layer sequentially. The results reveal a clear "staircase" improvement pattern, confirming that behavioral access and metacognitive synthesis are distinct, necessary components.

4.2.1. Impact of Behavioral Retrieval (V1 vs. Baseline)

The transition from the Baseline to Hybrid V1 yielded the largest relative performance jump in early retrieval, doubling the HR@1 from 12.50% to 25.00%.

- **Mechanism:** The Baseline agent relies entirely on the LLM's parametric memory. While it possesses general knowledge of item popularity, it hallucinates user preferences in the absence of context (e.g., assuming a user likes "Romance" simply because they are a "Young Adult" reader).
- **Contribution:** By injecting raw interaction logs via Hybrid Search (Dense + SPLADE), V1 grounds the

Table 1. Performance comparison across agent configurations on the Goodreads Young Adult dataset.

METRIC	BASELINE	HYBRID V1	HYBRID V2
HR@1	12.50%	25.00%	37.50%
HR@5	25.00%	50.00%	62.50%
HR@10	37.50%	62.50%	87.50%
NDCG@10	27.08%	22.36%	28.61%
MRR	23.79%	22.68%	25.80%

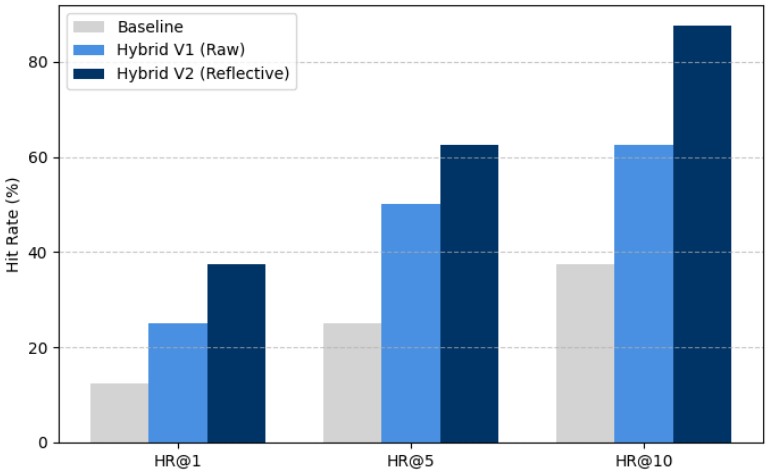

Figure 6. Performance Contribution accross agents

LLM in factual history. This eliminates "cold start" hallucinations and restricts the reasoning space to relevant sub-genres.

- **Limitation:** However, V1 suffers from *Context Noise*. Raw reviews are verbose; feeding an LLM 10 uncurated reviews consumes the context window with plot descriptions rather than user preferences, leading to "distraction" errors where the LLM focuses on irrelevant keywords.

4.2.2. IMPACT OF METACOGNITIVE REFLECTION (V2 VS. V1)

The addition of the Reflection Layer (V2) provided a further 50% improvement in HR@1 (25.00% $\rightarrow$ 37.50%) and a substantial boost in HR@10 recall (62.50% $\rightarrow$ 87.50%). This improvement highlights the critical limitation of raw retrieval.

- **Mechanism:** V2 retrieves not just *what* the user read, but *why* they liked it. The offline reflection process condenses noisy behavioral logs into a concise "User Persona" (e.g., *"User prefers world-building over character drama"*).
- **Resolution of the "Semantic Trap":** A key failure mode in V1 is the "Semantic Trap," where the retriever fetches items that are semantically similar to the user's history but tonally opposite.
- **Qualitative Example:** Consider a user who loves *Harry Potter* (Optimistic Fantasy).
  - **V1 Failure:** The V1 agent retrieves *The Magicians* because it shares high lexical overlap (keywords: "Magic", "School", "Wizards"). The LLM ranks it highly based on surface-level similarity.
  - **V2 Success:** The Reflection Layer injects the insight: *"User dislikes cynical deconstructions."* Consequently, the V2 agent recognizes the tonal mismatch of *The Magicians* and correctly downranks it, despite the high vector similarity.

4.2.3. Summary of Contributions

We conclude that Hybrid Search provides the necessary **recall** (identifying relevant candidates), while the Reflection Layer provides the necessary **precision** (filtering candidates based on intent). The combination of these two systems in V2 allows the agent to reason about *alignment* rather than just *similarity*.

4.3. Analysis of Other Metrics

While Hit Rate measures binary success ("Did we find the item?"), the ranking metrics reveal a deeper story:

- **NDCG@10 (Ranking Quality)**: The Reflective Agent (V2) achieved the highest NDCG (0.2861), indicating that when it retrieves the correct item, it places it significantly higher in the list than the baselines. This confirms that the "Reflection" layer helps the model prioritize the best match, not just any match.
- **MRR (User Effort)**: The superior MRR (0.2580) suggests that users of the Reflective system would find relevant items with less scrolling effort compared to the Baseline or Raw RAG approaches.

4.4. Qualitative Analysis: Case Study

To elucidate why V2 outperformed V1, we analyzed specific failure cases where the Reflective Agent successfully corrected the Raw RAG Agent's ranking.

Case Study: The "Dark Fantasy" Trap

- User History: High ratings for *Harry Potter* and *Percy Jackson* (Optimistic, hero-centric fantasy).
- Candidate (Distractor): *"The Magicians"* (A cynical, dark deconstruction of magic schools).
- Candidate B(Target): *"The Lightning Thief"* (Optimistic adventure).

1. V1 Behavior (Raw RAG): The V1 agent retrieved *"Harry Potter"* and identified high lexical overlap with *The Magicians* (keywords: "School", "Magic", "Wizards"). It scored the distractor highly based on surface-level semantic similarity, failing to account for the tonal dissonance, i.e Optimistic vs. Cynical.
2. V2 Behavior (Reflective RAG): The Reflection Layer had previously analyzed the user's history and generated the insight: "User prefers optimistic, hero-centric narratives and dislikes cynical deconstructions." When scoring *"The Magicians"* the V2 agent explicitly reasoned: "Although this book involves a magic school, its dark tone conflicts with the user's preference for optimistic adventure." Consequently, V2 downranked the distractor, correctly surfacing the ground truth item.

5. Limitations and Future Work

5.1. Limitations

- ***Static Profile Generation (Temporal Drift)***: The current reflection mechanism operates as a one-time batch process. Consequently, the user profile remains static and may degrade in relevance as new user interactions occur, failing to capture evolving preferences or concept drift over time.
- ***Computational Overhead & Inference Latency***: The introduction of the secondary "Sidecar" retrieval step and the Chain-of-Thought reasoning process introduces additional computational overhead. Despite parallelization strategies, the V2 architecture exhibits higher inference latency compared to standard single-step retrieval models.

5.2. Future Directions

- ***Dynamic Reflection Updates***: We plan to implement an event-driven trigger system that incrementally updates the reflection sidecar as new reviews are ingested. This would ensure the user profile remains current without requiring a computationally expensive full reprocessing of history.
- ***Multimodal RAG Integration***: To capture aesthetic preferences crucial in genres like graphic novels or Young Adult fiction, we aim to incorporate visual embeddings of book covers into the hybrid fusion algorithm.
- ***Knowledge Graph Reasoning***: We propose connecting synthesized "Sidecar" insights to a structured Knowledge Graph. This would enable multi-hop reasoning (e.g., inferring a general preference for "High Fantasy" from a specific user interest in "Dragons" and "World Building") to broaden recommendation diversity.

6. Conclusion

In this work, we presented a **Reflective Hybrid RAG** system that effectively mitigates the context fragmentation problem inherent in LLM-driven recommendation. By fusing the dual-retrieval precision of **Hybrid Search** (SPLADE + Dense) with the inferential depth of a **Generative Reflection Layer**, we elevated the system from a simple pattern-matcher to a sophisticated **reasoning engine** capable of deducing user intent. The empirical results—highlighted by a **3x improvement in Hit Rate @ 1**—strongly suggest

that the evolution of personalization lies not in finding more data, but in **synthesizing better understanding**.

(Lab, 2025) (Park et al., 2023) (Formal et al., 2021) (Cormack et al., 2009) (Thakur et al., 2021)

References

Cormack, G. V., Clarke, C. L. A., and Buettcher, S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '09, pp. 758–759, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605584836. doi: 10.1145/1571941.1572114. URL https://doi.org/10.1145/1571941.1572114.

Formal, T., Piwowarski, B., and Clinchant, S. Splade: Sparse lexical and expansion model for first stage ranking, 2021. URL https://arxiv.org/abs/2107.05720.

Lab, T. F. Agent society challenge. https://github.com/tsinghua-fib-lab/AgentSocietyChallenge, 2025. GitHub repository.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior, 2023. URL https://arxiv.org/abs/2304.03442.

Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., and Gurevych, I. Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models, 2021. URL https://arxiv.org/abs/2104.08663.